Item 2 - Practice - Accredited

Reminders

Typing text responses

Where you are instructed to type responses, use the files specified in the question text.

The margin characters, '>', indicate the maximum number of statements you can type.

Type at least nearly as many points as there are '>' characters to maximize your score.

Aim to keep statements brief and direct; prefer bullet points to longer sentences.

Case-sensitive by default

Unless stated otherwise, assume all instructions/examples are case-sensitive.

Canberra College IT Faculty

This means that 'A' is not the same as 'a'.

This can be relevant when referring to elements of source code or design parameters.

Text-Editing tools

You can open a text file in nano/vim by typing `nano <file>`/`vim <file>` in the terminal.

To open a file in kate via the terminal, type: `kate <file>`.

You can also open it by using the graphical application launcher in the corner.

Writing code

Aim to write the shortest amount of code which fulfills the specification.

Your code may be marked for readability and style.

Read the commented header of the response file for advice.

This is a practice paper

Some marking schemes are visible to inform your approach and response.

This item can only give you an idea of the format and tone of the actual item.

You should complete your class-based practice tasks and request feedback if necessary.

Canberra College IT Faculty

Reminders (Technology-Specific)

Basic Bash Commands Cheat Sheet

- `pwd` – Print the current working directory
- `ls` – List files and directories in the current location
- `cd <dir>` – Change to directory `<dir>`
- `mkdir <dir>` – Create a new directory named `<dir>`
- `rm <file>` – Remove a file
- `cp <src> <dest>` – Copy a file from `<src>` to `<dest>`
- `mv <src> <dest>` – Move or rename a file from `<src>` to `<dest>`
- `cat <file>` – Display the contents of a file
- `clear` – Clear the terminal screen

Canberra College IT Faculty

Executing Java programs

To compile and run your Java program, `YourFile.java`, from the terminal:

1. Navigate to the directory containing your `YourFile.java`
2. Compile it with `javac YourFile.java`
3. Run the compiled program with `java YourFile`

Working with Eclipse (Java IDE)

To create, edit, and run a simple Java program in Eclipse:

1. Open Eclipse and create a new Java Project via `File > New > Java Project`
2. Create a new Java class via `File > New > Class`. Include a `public static void main(String[ args)` method stub if you wish.
3. Type your program code into the editor.
4. Save your file with `Ctrl+S`.
5. Run the program with `Ctrl+F11` or via `Run > Run`.

Canberra College IT Faculty

Java documentation help

You do not need to memorize Classes!

- Hover your mouse cursor over any standard class to read Java documentation about it.
- You can view Java documentation in a separate pane: `Window > Show View > Javadoc`

Canberra College IT Faculty

Submitting Java files

Named submission files

Save your responses to the provided, named files, for example `Question1.java`, unless the question says otherwise.

Only those named files will be collected from your machine.

If a required file is renamed, moved out of its `Item` directory, or cannot be opened by markers, you risk a zero for that question or at least significant penalties.

Your responsibility: correct name, correct location, compilable content.

Provided class stub

For convenience, you may be provided a class stub with an empty `main` method.

You can compile and run this as-is, then add and run your own code.

Make sure to comment out any test code before submission.

```java
public class Question1 {
    public static void main(String[] args) {
        // This stub has been added for your convenience
    }
}
```

Canberra College IT Faculty

Defining an additional required class in the same file

If a question requires you to define a specific class in the same file, for example `Foo`, make it package-private.

That means using no access modifier, so the public class name still matches the filename.

Example:

Canberra College IT Faculty

```java
public class Question1 {
    public static void main(String[] args) {
        // This stub has been added for your convenience
    }
}

class Foo {
```

```
    private int bar;
    public String baz;
}
```

Canberra College IT Faculty

Canberra College IT Faculty

Canberra College IT Faculty

Jump To

Q1 (2 parts) 7

Q2 (2 parts) 8

Q3 (2 parts) 10

Q4 (2 parts) 11

Q5 (2 parts) 12

Any fool can write code that a computer can understand. Good programmers write code that humans can understand.

— Martin Fowler

First, solve the problem. Then, write the code.

— John Johnson

Canberra College IT Faculty

Q1 (2 parts)



a) A garden coordinator stores plot-preparation percentages in an `ArrayList<Integer>`. Write `int countReadyPlots(ArrayList<Integer> percentages)` to return how many values are at least `80`. Call the method from `main` with `75, 80, 95, 60`, then print the ready count and the number of values checked.

The following is an **example** of correct program output.
This program must not request input during execution.
Your program may be tested with other values of the same type(s).

```
Ready plots: 2
Values checked: 4
```

Note: The method must also work with other integer lists.

Save your source code in `Question1a.java`.

Canberra College IT Faculty

b) The coordinator is deciding whether to store a fixed twelve-month rainfall record in an array or a changing list of active garden plots in an `ArrayList`. Analyse the two choices.

- Recommend the suitable structure for each dataset.
- Justify both recommendations using size, access, and update requirements.

Save your response in `q1_response.txt`.

Canberra College IT Faculty

Canberra College IT Faculty

Q2 (2 parts)

a) A booking app must keep asking until the user enters a whole-number group size from 1 to 12. Write `int readGroupSize(Scanner input)` using `try`, `catch`, and `finally`. Invalid text or an out-of-range number must produce distinct feedback; the `finally` block must print `Attempt handled` after every attempt. Return the first valid value.

Canberra College IT Faculty

The following is an **example** interaction.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

Do not assume that only inputs of the type shown will be used.

```
Enter group size:  four
Invalid integer
Attempt handled
Enter group size:  15
Group size must be from 1 to 12
Attempt handled
Enter group size:  4
Attempt handled
```

Note: The marker may test a different input sequence.

Save your source code in `Question2a.java`.

b) The booking app represents one scheduled workshop as an object. Implement a Java class `Workshop` with private `title` and `capacity` fields, a constructor, and `String summary()`. In `main`, create two different workshop objects and print both summaries.

The following is an **example** of correct program output.

This program must not request input during execution.

Your program may be tested with other values of the same type(s).

```
Java Basics - capacity 12
Web Design - capacity 18
```

Canberra College IT Faculty

Note: The exact workshop names and capacities may differ.

Save your source code in `Question2b.java`.

Canberra College IT Faculty

Q3 (2 parts)

a) A training-centre booking app models each workshop with a fixed capacity and a current enrolment. Implement a self-contained `Java` class `Workshop` with a private `enrolled` field, `boolean hasSpace()`, and `boolean enrol()`. The enrolment count starts at zero. Calling `enrol()` must change the count only when space remains and report whether the state changed.

Save your source code in `Question3a.java`.

Canberra College IT Faculty

Canberra College IT Faculty

b) Continue with the training-centre booking app from part a. The app introduces `OnlineWorkshop` and `LabWorkshop` as specialised forms of `Workshop`. Both override `String accessInstructions()`. Analyse this inheritance design.

- Specify the state and behaviour that should remain in `Workshop`.
- Justify the overridden behaviour in each subtype.
- Evaluate how a collection of `Workshop` references supports future workshop types.

Save your response in `q3_response.txt`.

Q4 (2 parts)



a) A community media-library app manages printable resources. Every resource has `String label();` file-backed resources also need stored `filename` state. Analyse an extensible design using an interface and an abstract class.

- Define the contract owned by `PrintableResource`.
- Design an abstract `FileResource` responsibility without duplicating subtype work.
- Evaluate one benefit and one limitation of combining these abstractions.

Save your response in `q4_response.txt`.

b) Continue with the media-library app from part a. Implement a `Java` class `ResourceCode` that overrides `toString()` to return its code. Add a non-instantiable utility class `ResourceTools` with static methods `boolean sameCode(ResourceCode first, ResourceCode second)` and `int countActive(boolean[] states)`. Demonstrate both methods from `main`.

Save your source code in `Question4b.java`.

Canberra College IT Faculty

Canberra College IT Faculty

Q5 (2 parts)



a) A report-processing app represents one export job with states `QUEUED`, `RUNNING`, and `COMPLETE`. It also needs a worker that accesses one enclosing job's state. Design the type structure.

- Choose whether the state type and worker should be static nested or inner members.
- Justify both choices using ownership, instance access, and coupling.
- Evaluate one maintenance risk in the design.

Save your response in `q5_response.txt`.

b) Continue with the report-processing app from part a. Implement `ReportExporter` with an enum `ExportState` containing `QUEUED`, `RUNNING`, and `COMPLETE`. Store the current state in the enclosing object. Add an inner class implementing `Runnable`; its `run()` method changes the state to `RUNNING`, prints `Exporting`, then changes it to `COMPLETE`. Start the worker in a new `Thread`, wait with `join()`, and print the final state.

Save your source code in `Question5b.java`.

Canberra College IT Faculty

Canberra College IT Faculty

Canberra College IT Faculty



Canberra College IT Faculty

"Nighttime Stories"

— xkcd by Randall Munroe